Sketches de firmware TeleCon

Qué `.ino` grabar

ESP32 DevKit por Bluetooth, y ESP32-CAM para video

Versión 1.0 2026

Resumen

La aplicación Android descarga un ZIP por cada conexión desde el repositorio de firmware. Cada ZIP es un sketch de Arduino, y el sketch es el `.ino` del mismo nombre. Esta nota nombra cada sketch de control por Bluetooth y cada sketch de ESP32-CAM, y las dos formas de añadir una cámara.

DevKit + Bluetooth CAM solo video + DevKit

Índice

1. Dos formas de añadir una cámara **1**

2. Control Bluetooth en una ESP32 DevKit **2**

3. ESP32-CAM, solo video **3**

4. Qué contiene el ZIP **3**

1. Dos formas de añadir una cámara

Las palancas, los botones y la telemetría siempre necesitan un sketch de control. El video es opcional y usa una ESP32-CAM. Elija uno de estos montajes.

Montaje	Placas	Sketches
Solo Bluetooth	Una ESP32 DevKit	Uno de los tres `.ino` de Bluetooth
Solo cámara, más Bluetooth	ESP32-CAM para video, DevKit para las palancas	`TeleCon_CAM_SoftAP_Video.ino` en la CAM, y un sketch Bluetooth en la DevKit

El teléfono se une al SoftAP de la cámara para la imagen. No envía las palancas por ese enlace de video. Las palancas van al sketch de control: Bluetooth en la DevKit.

2. Control Bluetooth en una ESP32 DevKit

Placa en la aplicación: **DevKit**. Esta placa no lleva cámara. Abra la carpeta del sketch en el IDE de Arduino (la carpeta que contiene el `.ino`) y súbalo a un módulo ESP32 Dev.

ZIP / `.ino`	Ajuste en el teléfono	En el aire
`TeleCon_Classic_Simpl`	Classic + Simple	Nombre `ESP32-TC-RC-BT-Simple`, PIN `1234`. Saludo `proto,simple`. Líneas de texto `RC:CTRL`, `RC:BTN`, `RC:DATA`, `RC:PLOT`.
`TeleCon_Classic_Binar`	Classic + Binary	Nombre `ESP32-TC-RC-BT-Binary`. Saludo `proto,binary`. Tramas `AA 55`, `BB 66`, `CC 11/22/33`.
`TeleCon_BLE_Binary`	BLE Binary	Nombre `ESP32-TC-RC-BLE-Binary` (Nordic UART). Las mismas tramas binarias que Classic Binary.

`TeleCon_Classic_Simple.ino` Un solo archivo para la primera placa. Ponga el código de motores y servos en `onControl()` y `onButton()`, donde el archivo dice `YOUR CODE HERE`. El sketch ya incluye un servo de dirección en el GPIO 13: la palanca `rx` = 0 es un pulso de 1500 µs. Los ocho bytes de telemetría (CH1–CH8 en el teléfono) se eligen en `kTelemetryChannels[]` dentro de este mismo archivo. Con `ENABLE_PLOT` en 1 la placa envía `RC:PLOT` cada 50 ms aproximadamente.

`TeleCon_Classic_Binary.ino` El `.ino` solo arranca el Bluetooth y los módulos del vehículo. Los pines, el PWM y los ocho canales de telemetría están en los otros archivos de esa carpeta. Edite `pins.h` para su controlador y `TelemetryChannels.h` para lo que informan CH1–CH8. El ajuste del teléfono debe ser Classic + Binary antes de conectar. El formato del cable es el documento del protocolo de control, no las líneas de texto del sketch Simple.

`TeleCon_BLE_Binary.ino` Los mismos módulos de vehículo y la misma tabla `TelemetryChannels.h` que Classic Binary. El enlace es BLE en lugar de Classic SPP. Elija BLE Binary en la aplicación. No lo empareje como un dispositivo Classic.

Estos tres sketches son también la mitad de control del montaje de dos placas que sigue. Grabe solo uno de ellos en la DevKit.

3. ESP32-CAM, solo video

Úselo cuando una DevKit ya ejecuta uno de los sketches Bluetooth y quiere imagen desde una segunda placa.

`TeleCon_CAM_SoftAP_Video.ino` ZIP `TeleCon_CAM_SoftAP_Video.zip`. Grábelo en una ESP32-CAM (AI-Thinker). Este sketch sirve video y no acepta palancas.

- SoftAP `TeleCon-RC-CAM-Starter`, contraseña `telecon1234`, dirección `192.168.4.1`.
- Imagen: `http://192.168.4.1/stream` (MJPEG) o `/capture` (un JPEG).
- Calidad del flujo: `http://192.168.4.1/camconfig`.
- No hay puerto de control. Esta placa no recibe `RC:CONNECT`.

En el teléfono, active la cámara superpuesta y deje la conexión Bluetooth apuntada a la DevKit (Classic Simple, Classic Binary o BLE Binary). El teléfono usa Wi-Fi para la CAM y Bluetooth para las palancas al mismo tiempo. El control de las palancas no va en la ESP32-CAM.

4. Qué contiene el ZIP

La aplicación descarga `zips/TeleCon_<nombre>.zip`. Descomprímalo y abra la carpeta que contiene el `.ino` del mismo nombre. Arduino compila cada `.cpp` y `.h` de esa carpeta, así que un sketch binario es el `.ino` más esos módulos. No vuelva a copiar `SteerCenter`: los sketches de serie no guardan un centro de dirección. El recorte queda en el teléfono, y `rx` $= 0$ es $1500\,\mu s$.